RAG Assignment: Baseline vs. Re-Ranking

Harshita Kumari

GitHub: `https://github.com/Harshita-K/RAG`

19 October 2026

Setup

All code is in a single notebook, `rag_reranking.ipynb`, in the repository linked above.

- **Document:** GDPR, Regulation (EU) 2016/679, official 88-page PDF. Chosen for its exceptions ("shall not apply where…"), prohibitions and heavily repeated terms (consent, controller, processing).
- **Chunking:** 478 chunks of ≤1,000 characters (∼220 tokens); never crossing an article boundary; article/recital heading prepended; footnotes and page headers removed.
- **Bi-Encoder:** `sentence-transformers/all-MiniLM-L6-v2`, FAISS `IndexFlatIP` on normalized vectors (= cosine similarity).
- **Cross-Encoder:** `cross-encoder/ms-marco-MiniLM-L6-v2`.
- **LLM:** `Qwen2.5-3B-Instruct` run locally, greedy decoding, identical prompt for Answer 1 and Answer 2.
- **Hardware:** Apple M1, 16 GB. Retrieval models on CPU, LLM on GPU (MPS).

Queries and Evaluation Method

Observations

- We first read through the regulation and identified the tricky parts, such as exceptions, "shall not" clauses, and legal terms that have simpler everyday equivalents. Based on these, we drafted around 40 candidate questions, tested both retrievers, and selected the 8 questions that showed the clearest differences.
- The queries use everyday language rather than the exact terms used in the regulation. For example, "computer algorithm" is used instead of "solely on automated processing", and "fired" instead of "dismissed". This helps test whether the retrievers can find relevant information even when the wording does not match directly.
- Each query has a gold phrase that appears only in the chunk or chunks containing the answer. We use text matching to identify these gold chunks instead of assigning their IDs manually. A query is counted as answered only when at least one gold chunk appears in the top 3 results.
- Q1–Q6 focus on the failure cases described in the brief, while Q7–Q8 are included as counter-examples to show situations where re-ranking actually makes the results worse.
- The prompt instructs the LLM to answer using only the provided context and to respond with "Not found in the provided context" when the information is missing. This helps identify cases where the retriever fails to find the relevant information, rather than allowing the model to fill in the gaps using its own knowledge.

- Since the queries were deliberately selected to highlight specific effects, the result of 6 out of 8 should not be treated as a general measure of how well re-ranking performs.

1 Architecture Comparison

	Bi-Encoder	**Cross-Encoder**
Input	Query and chunk encoded **separately**	Query and chunk fed in **together** as one sequence
Output	One vector each (384 dimensions here)	One relevance score per pair
Relevance measure	Cosine similarity between two vectors	Learned score from joint self-attention across all tokens
Pre-computable?	Yes, chunk vectors computed once, offline	No, every pair computed at query time
Cost per query	1 model pass + vector search	1 model pass **per candidate chunk**
Weakness	Compresses meaning into one vector: blurs negation, rewards shared keywords	Too slow to run over a whole corpus

Table 1: Bi-Encoder vs. Cross-Encoder.

Observations

- The Bi-Encoder processes the query and chunk separately, so it can identify texts that are related but cannot directly determine whether a chunk answers the query.
- The Cross-Encoder considers the query and chunk together, allowing it to understand their relationship more accurately. For example, it can connect "fired" with "shall not be dismissed" (Q6).
- This is why a two-stage approach is useful: the Bi-Encoder quickly retrieves 10 relevant candidates, and the Cross-Encoder ranks them to select the best 3.

2 Qualitative & Rank Shift Analysis

The chunk containing the answer reached the LLM in **1 of 8** queries with the baseline and **6 of 8** after re-ranking.

2.1 Rank changes

ID	Query	Failure type	Critical chunk	Rank shift
Q1	Does the GDPR apply to someone keeping a personal address book?	Keyword trap	16, Recital 18	#8 → #1
Q2	Can a decision about me be made only by a computer algorithm?	Synonym	276, Art. 22(1)	#4 → #1
Q3	When does a company not need to tell affected users about a data breach?	Exception	305, Art. 34(3)	#5 → #3
Q4	Is a controller exempt from informing people when it got their data from someone else and contacting them would take disproportionate effort?	Exception	258, Art. 14(5)	#4 → #2
Q5	Which public bodies are not regarded as recipients of personal data?	Negation	217, Art. 4(9)	#10 → #2
Q6	Can the data protection officer be fired for doing their job?	Negation / role limit	320, Art. 38(3)	#6 → #1
Q7	Is an employer prohibited from processing health data about employees?	Counter-example	237, Art. 9(2)(b)	#4 → #9
Q8	Is processing allowed to save someone's life?	Counter-example	228, Art. 6(1)(d)	#2 → #5

Table 2: Rank of the chunk that answers each query (only ranks 1–3 reach the LLM).

2.2 Why the ranks changed: scores

	Bi-Encoder (cosine)				Cross-Encoder (logit)	
ID	Gold	Rank-1	Gap	Top-10 spread	Gold	Top-10 spread
Q1	0.421	0.478	0.057	0.064	−4.08	7.17
Q2	0.365	0.425	0.060	0.107	−5.15	6.00
Q3	0.614	0.630	0.016	0.094	0.55	6.07
Q4	0.605	0.637	0.032	0.066	1.28	7.71
Q5	0.568	0.638	0.070	0.070	6.72	11.05
Q6	0.509	0.621	0.112	0.159	2.59	8.97
Q7	0.541	0.624	0.083	0.103	−3.07	8.37
Q8	0.489	0.491	0.002	0.075	−4.01	8.75

Table 3: Score of the gold chunk vs. the Bi-Encoder's rank-1 chunk, and the spread (max − min) of scores over the 10 candidates. The two scales are not directly comparable; what matters is how well each separates the candidates.

Observations

- The Bi-Encoder gives all 10 candidates very similar scores (spread 0.06–0.16), with the gold chunk just 0.02–0.11 behind the top-ranked candidate. This makes it difficult to tell how relevant each chunk actually is from the scores alone.
- Q1 is the best example: all 10 candidates have scores within a range of just 0.064, but the Cross-Encoder gives the gold chunk a score of −4.08 and the next best chunk −10.99, creating a clear gap of almost 7 points.

- In Q5, the gold chunk ranked last (#10) with the Bi-Encoder, even though its score was only 0.070 below the top-ranked candidate.

2.3 Answer 1 vs. Answer 2

ID	Answer 1 (baseline)	Answer 2 (re-ranked)	Verdict
Q1	"we cannot determine whether the GDPR applies"	Not applicable to "purely personal or household activities" (Recital 18)	Baseline **omitted** the exemption; fixed
Q2	Correct: another Art. 22 chunk was already in its top 3	Correct, cites Art. 22(1) directly	Rank improved, answer already fine
Q3	Uses "unlikely to result in a risk", the test for notifying the *authority* (Art. 33), not users	Gives the encryption exemption, Art. 34(3)(a)	Baseline **misstated the rule**; Answer 2 correct but omits 34(3)(b),(c)
Q4	Claims the controller "may charge a reasonable fee or refuse to act" (rules for excessive requests)	Answers from Recital 57 instead of the Art. 14(5) chunk it was given	Baseline **hallucinated**; Answer 2 still wrong: LLM failure, not retrieval
Q5	"Not found in the provided context"	Authorities receiving data for "a particular inquiry" are not recipients	Baseline **omitted**; fixed
Q6	"no explicit mention of whether the DPO can be fired"	"shall not be dismissed or penalised ... for performing his tasks"	Baseline **omitted** the clause; fixed
Q7	"no explicit prohibitions" (wrong: Art. 9(1) prohibits by default)	Same claim; the employment exception left the context	Both **misstate**; re-ranking made context worse
Q8	Correct via Recital 46	Correct via Recital 46	Rank dropped but no harm

Table 4: LLM answers with baseline vs. re-ranked context (Qwen2.5-3B, identical prompt).

Observations

- Q1, Q5, Q6: the baseline ranked chunks that share words with the question (information duties, DPO tasks) above the chunk that answers it.
- Q3, Q4: the general rule ranks high; the exception sits in a separate chunk lower down (the "except when..." failure).
- Q4: correct retrieval does not guarantee a correct answer; a 3B model can still read the wrong passage.
- Q7, Q8: the Cross-Encoder (trained on MS MARCO web search) preferred explanatory recitals over terse article text — a likely explanation, not tested. Re-ranking is not uniformly better.

3 Latency Profiling

Stage	Pipeline 1 (ms)	Pipeline 2 (ms)
Query embedding (Bi-Encoder)	5.9 ± 0.4	5.9 ± 0.4
Vector search (FAISS, top 10)	0.04 ± 0.02	0.04 ± 0.02
Cross-Encoder re-rank (10 pairs)	–	117.6 ± 23.1
Retrieval total	**5.9**	**123.5**
LLM generation (Qwen2.5-3B)	38,271 ± 9,986	34,858 ± 6,937
End-to-end	**38,277**	**34,982**

Table 5: Mean ± std of 40 runs (8 queries × 5 repeats, first cold run per query dropped); retrieval on CPU, Apple M1. LLM time is one run per query per pipeline.

Observations

- Re-ranking makes retrieval ∼20× slower (5.9 → 123.5 ms) but is only ∼0.3% of end-to-end time, because generation takes ∼35 s.
- The Cross-Encoder costs ∼12 ms per (query, chunk) pair, so its cost grows with the number of candidates.
- Pipeline 2's slightly lower LLM time is noise from answer length, not an effect of re-ranking.

4 Scaling Question

Corpus size (chunks)	Cross-Encoder over everything	Two-stage pipeline
478 (this PDF, measured)	7.5 s	0.12 s
10,000	2.6 min	0.12 s
1,000,000	4.4 hours	0.12 s
100,000,000	18.1 days	0.12 s

Table 6: Time per query. Measured 15.7 ms per (query, chunk) pair on CPU, extrapolated linearly.

Observations

- A Cross-Encoder must run once per (query, chunk) pair at query time and nothing can be pre-computed, so its cost grows with the whole corpus on every query.
- Bi-Encoder cost is paid once: chunk vectors are computed offline; a query costs one encoding pass plus a vector search (0.04 ms here; ANN indexes like HNSW keep this fast at millions of vectors).
- Two-stage keeps most of the benefit: the Bi-Encoder's top 10 already contained the right chunk in all 8 queries; only its ordering was wrong.
- A full Cross-Encoder pass over all 478 chunks did not rank better here: Q3 and Q4 landed at #4 and #3 instead of #3 and #2, and Q7 at #22.

Conclusion

Observations

- Both pipelines use the same corpus, top 10 candidates, prompt, and LLM, differing only in the re-ranking step. Re-ranking brought the correct answering chunk into the LLM context for 6 out of 8 queries, compared to just 1 out of 8 with the baseline.
- For 4 of these 6 queries (Q1, Q3, Q5, Q6), the baseline answer missed or incorrectly stated the rule, while Answer 2 corrected it. Q4 also shows that even with the right chunk retrieved, the LLM can still give a wrong answer if it focuses on the wrong passage.
- Re-ranking did not improve every query. For Q7 and Q8, it replaced article text with recitals, and in Q7 this made the context worse, with both answers remaining incorrect.
- The main reason for the improvement is that the Bi-Encoder struggles to distinguish chunks that discuss similar topics. The Cross-Encoder compares the query and chunk together, making it easier to identify which chunk is actually relevant.
- Re-ranking adds around 118 ms per query, which is only about 0.3% of the total response time. This makes the two-stage approach worthwhile here, whereas running the Cross-Encoder on the entire corpus would be too expensive.
- With more time, the evaluation could be improved by testing a larger and more diverse set of queries, experimenting with different chunk sizes, and evaluating the generated answers directly instead of focusing only on the rank of a single gold chunk.

Reproducing

```
pip install -r requirements.txt
jupyter notebook rag_reranking.ipynb   # models download on first run
```

All tables in this report are written by the notebook to `results/`.